



华北电力大学

毕业设计（论文）

论文题目： 基于多智能体强化学习的2D竞技场博弈对抗研究

院 系： 控计学院

专 业： 计算机科学与技术

班 级： 计算2202

姓 名： 蒋伦吉

学 号： 120221080514

指导教师： 刘春阳

2026年5月

摘 要

多智能体强化学习（Multi-Agent Reinforcement Learning, MARL）致力于解决多个自主智能体在共享环境中通过交互学习最优策略的问题，在实时策略博弈、机器人集群协作等复杂决策场景中具有重要的研究价值与应用前景。

本文基于 Godot 游戏引擎构建了 2D 俯视角多智能体竞技场博弈环境《Tiny Swords》，设计了包含自身状态、对手信息、实体感知与 LiDAR 射线检测的 174 维分段式观测空间，以及基于势能的奖励塑形（Potential-Based Reward Shaping, PBRS）系统，涵盖奖励球引导、障碍物避障与差异化事件奖励。采用独立近端策略优化（Independent PPO, IPPO）算法，通过课程学习与大量消融实验，系统研究了势函数形式、障碍物处理策略与网络架构对智能体基础行为的影响。

在此基础上，针对多智能体训练中的非平稳性问题，本文设计了迭代自博弈训练框架：各智能体独立训练至收敛后，采用冻结对手、轮换微调的方式驱动策略持续对抗进化，并引入对手池管理与 ELO 评分机制追踪策略强度变迁。实验结果表明，迭代自博弈能够有效促使智能体涌现出策略循环与行为分化，验证了该方法在驱动多智能体博弈对抗中的有效性。

关键词：多智能体强化学习；独立近端策略优化；博弈对抗；奖励塑形；循环神经网络

ABSTRACT

This is the English abstract. It should correspond to the Chinese abstract, with approximately the same content and length.

Multi-Agent Reinforcement Learning (MARL) addresses the problem of multiple autonomous agents learning optimal policies through interaction in shared environments, with significant research value and application prospects in complex decision-making scenarios such as real-time strategy games and robot swarm coordination.

This paper constructs a 2D top-down multi-agent arena game environment called “Tiny Swords” based on the Godot game engine. A 174-dimensional segmented observation space encompassing self-state, opponent information, entity perception, and LiDAR ray detection is designed, along with a Potential-Based Reward Shaping (PBRs) system covering ball collection guidance, obstacle avoidance, and differentiated event rewards. Using the Independent Proximal Policy Optimization (IPPO) algorithm, the influence of potential function forms, obstacle handling strategies, and network architectures on agent behavior is systematically studied through curriculum learning and extensive ablation experiments.

To address the non-stationarity problem in multi-agent training, this paper proposes an iterative self-play training framework: after independent training to convergence, agents are sequentially fine-tuned against frozen opponents, driving continuous competitive strategy evolution. An opponent pool management and Elo rating mechanism is introduced to track strategy strength dynamics. Experimental results demonstrate that iterative self-play effectively drives agents to exhibit strategy cycles and behavioral differentiation, validating the effectiveness of the proposed approach in multi-agent competitive scenarios.

KEY WORDS: Multi-Agent Reinforcement Learning; Independent PPO; Game-Theoretic Adversarial; Reward Shaping; Recurrent Neural Network

目 录

目 录

前言	1
1 绪论	1
1.1 研究背景与意义	1
1.2 国内外研究现状	2
1.3 研究内容与创新点	2
1.4 论文组织结构	2
2 相关理论与技术基础	3
2.1 强化学习基础	3
2.2 多智能体强化学习	4
2.3 奖励塑形理论	4
2.4 Godot 引擎与 RL 集成框架	5
3 博弈环境设计与实现	5
3.1 游戏总体设计	5
3.2 观测空间设计	6
3.3 奖励函数设计	6
3.4 课程学习环境	7
4 单智能体基础训练与消融实验	7
4.1 训练框架与超参数	7
4.2 奖励球势函数对比	8
4.3 障碍物避障策略	9
4.4 网络架构对比	9
4.5 观测空间消融：有效掩码	10
4.6 课程学习效果	10
5 多智能体迭代自博弈	10
5.1 多智能体训练的非平稳性分析	11
5.2 迭代自博弈训练框架	11
5.3 对手池管理与 ELO 评分	12
5.4 策略循环与对抗行为分析	12
5.5 实验结果与讨论	13
6 总结与展望	14
6.1 工作总结	14
6.2 未来展望	14
结论	15
A 附录A 核心算法伪代码	17
B 附录B 实验补充数据	18
致谢	19

前言

前言部分说明研究背景、目的和意义，介绍论文的主要研究内容和结构安排。注意前言不编"第*章"号。

近年来，人工智能技术在实时博弈对抗领域的应用日益广泛，从 AlphaGo 在围棋中超越人类顶尖棋手，到 AlphaStar 在星际争霸II中达到宗师级水平，再到 OpenAI Five 在 Dota 2 中击败世界冠军队伍，强化学习（Reinforcement Learning, RL）已展现出在复杂决策问题中的巨大潜力^[1]。

多智能体强化学习（MARL）将 RL 框架拓展至多个智能体共享环境的场景，其核心挑战在于环境的非平稳性（non-stationarity）：每个智能体的策略变化都会改变其他智能体所面对的环境动态，使得单智能体 RL 的理论保证不再直接适用。在博弈对抗场景中，智能体不仅需要学会利用环境中的资源，更需要学会感知对手策略并作出适应性调整，这正是本课题关注的核心问题。

本文围绕"构建博弈环境、训练基础策略、驱动策略对抗进化"三个层次展开研究。首先，基于 Godot 引擎开发了《Tiny Swords》2D 竞技场多智能体博弈环境，设计了高维分段观测空间与多层奖励塑形系统；其次，采用 IPPO 算法训练智能体掌握吃球、战斗、避障等基础行为，通过势函数对比、网络架构消融等实验确定最优配置；最后，提出迭代自博弈训练框架，驱动智能体在持续对抗中涌现出策略循环与行为分化。

第1章 绪论

1.1 研究背景与意义

博弈对抗是人工智能领域的核心挑战之一。从早期的国际象棋程序 Deep Blue，到近年来的 AlphaGo、AlphaStar、OpenAI Five，AI 在博弈领域的每一次突破都标志着智能决策技术的重大进步。与完全信息的棋盘博弈不同，实时策略博弈具有状态空间庞大、信息不完全、决策异步等特点，更贴近真实世界的决策场景。

多智能体强化学习为博弈对抗提供了一套端到端的学习框架。通过让智能体在环境中反复试错，RL 能够自动发现超越人类先验知识的策略。然而，在 MARL 场景下，环境非平稳性、信用分配（credit assignment）和探索-利用权衡等问题相互交织，给训练带来了额外的挑战。

本文旨在构建一个可控的 2D 多智能体博弈实验平台，系统研究从环境设计、奖励工程到策略对抗进化的完整流程，为多智能体博弈对抗的理论研究提供实践经验与实验支撑。

1.2 国内外研究现状

MARL 的研究可大致分为三类范式：集中训练集中执行（CTCE）、集中训练分散执行（CTDE）和分散训练分散执行（DTDE）。

在 CTDE 范式下，MADDPG^[2]通过集中 Critic 利用全局信息、分散 Actor 仅依赖局部观测，有效缓解了非平稳性问题；QMIX^[3]和 VDN^[4]通过值函数分解实现了多智能体协作。在 DTDE 范式下，IPPO^[5]将 PPO 独立应用于每个智能体，以其简洁性和实用性在多个 MARL 基准上取得了有竞争力的结果。

自博弈（self-play）是驱动策略进化的重要范式。AlphaGo 通过自我对弈生成训练数据不断精进棋力；AlphaStar 引入了联盟训练（league training），维护多个历史策略版本的手池，有效防止策略坍塌。Fictitious Play 和 Prioritized Fictitious Self-Play（PFSP）则为自博弈提供了博弈论基础。

在奖励工程方面，势能奖励塑形（PBRs）^[6]通过引入势函数为状态空间赋予先验知识，在保持最优策略不变的前提下加速学习。在 MARL 中，奖励设计还需考虑智能体间的激励相容问题。

1.3 研究内容与创新点

本文的主要研究内容与创新点包括：

（1）构建了基于 Godot 引擎的 2D 多智能体竞技场博弈环境《Tiny Swords》，设计了 174 维分段式观测空间与 LiDAR 射线检测机制，实现了 4 人混战游戏模式，支持课程学习与 per-player 差异化奖励配置。

（2）设计了包含奖励球势能引导、障碍物势能避障与事件稀疏奖励的多层 PBRs 奖励系统，通过系统的消融实验确定了最优的势函数形式（线性函数、最近球模式）与障碍物处理策略（稀疏惩罚 0.5 + 24 条射线）。

（3）对比了 MLP、Segmented MLP 与 GRU-MLP 三种网络架构在吃球任务与战斗任务中的性能差异，为观测空间的编码方式选择提供了实验依据。

（4）提出了迭代自博弈训练框架，通过冻结对手策略、轮换微调的方式驱动智能体在竞争中持续进化，引入对手池管理与 ELO 评分机制追踪策略强度变迁，实验观察到策略循环与行为分化现象。

1.4 论文组织结构

本文共分为六章，各章内容安排如下：

第一章介绍研究背景、国内外研究现状、研究内容与论文组织结构。

第二章阐述相关理论与技术基础，包括强化学习（MDP、Bellman 方程、PPO）、多智能体强化学习（IPPO、非平稳性）、奖励塑形理论（PBRs）与 Godot-RL 集成框架。

第三章详细描述博弈环境的设计与实现，包括游戏规则、观测空间、奖励函数与课程

学习环境的完整设计。

第四章介绍单智能体基础训练与消融实验，涵盖奖励球势函数对比、障碍物避障策略、网络架构对比与观测空间消融等实验。

第五章阐述多智能体迭代自博弈训练框架，分析策略循环与对抗行为的涌现。

第六章总结全文工作，展望未来研究方向。

第2章 相关理论与技术基础

2.1 强化学习基础

强化学习解决的是智能体在环境中通过试错学习最优策略的问题，其数学基础是马尔可夫决策过程（Markov Decision Process, MDP）。MDP 由状态空间 $\mathcal{S}$ 、动作空间 $\mathcal{A}$ 、状态转移概率 $P(s'|s,a)$ 和奖励函数 $R(s,a)$ 构成。智能体的目标是最大化累计折扣回报：

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \quad (2-1)$$

值函数 $V^\pi(s)$ 表示从状态 s 出发，遵循策略 π 的期望回报；动作值函数 $Q^\pi(s,a)$ 则表示在状态 s 采取动作 a 后的期望回报。最优值函数满足贝尔曼最优方程：

$$V^*(s) = \max_a \left[R(s,a) + \gamma \sum_{s'} P(s'|s,a) V^*(s') \right] \quad (2-2)$$

时序差分（Temporal Difference, TD）学习结合了动态规划与蒙特卡洛方法的优点，通过自举（bootstrapping）实现在线学习。

策略梯度方法直接对策略 π_θ 的参数 θ 进行优化，目标是最大化期望回报 $J(\theta) = \mathbb{E}_{\pi_\theta}[G_t]$ 。策略梯度定理给出：

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} [\nabla_\theta \log \pi_\theta(a|s) A^\pi(s,a)] \quad (2-3)$$

其中 $A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$ 为优势函数（Advantage Function）。

广义优势估计（Generalized Advantage Estimation, GAE）^[7] 通过 TD 误差的指数加权和来平衡估计偏差与方差：

$$\hat{A}_t^{\text{GAE}(\gamma,\lambda)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l} \quad (2-4)$$

其中 $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ 为单步 TD 误差。

近端策略优化（Proximal Policy Optimization, PPO）^[8] 通过剪切目标函数约束策略更

新幅度：

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t [\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t)] \quad (2-5)$$

其中 $r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ 为新旧策略的概率比。PPO 以其稳定性、样本效率和实现简洁性，成为当前应用最广泛的 RL 算法之一。

2.2 多智能体强化学习

多智能体强化学习将 MDP 扩展为马尔可夫博弈（Markov Game），定义为元组 $\langle N, \mathcal{S}, \{\mathcal{A}_i\}, P, \{R_i\}, \gamma \rangle$ ，其中 N 为智能体数量。每个智能体 i 的目标是最大化自身的期望累计回报。

MARL 面临的核心挑战是非平稳性（non-stationarity）：当所有智能体同时学习时，每个智能体面对的环境转移概率 $P(s'|s, a_1, \dots, a_N)$ 随其他智能体策略的变化而不断改变，破坏了单智能体 RL 中 MDP 的平稳性假设。

独立近端策略优化（Independent PPO, IPPO）是 MARL 中简洁而有效的 DTDE 方法。IPPO 为每个智能体维护独立的策略网络 π_{θ_i} 和价值网络 V_{ϕ_i} ，在训练期间仅使用各自的局部观测和奖励进行 PPO 更新，完全忽略其他智能体的存在：

$$L_i^{\text{IPPO}}(\theta_i) = \mathbb{E}_{(s_i, a_i, r_i, s'_i)} [L^{\text{CLIP}}(\theta_i; s_i, a_i, r_i, s'_i)] \quad (2-6)$$

尽管 IPPO 在理论上不处理非平稳性，但实证研究表明其在多个 MARL 基准上表现优异^[5]，特别是在各智能体目标具有冲突性的博弈对抗场景中。IPPO 的简洁性使其成为多智能体博弈研究的理想基线方法。

2.3 奖励塑形理论

奖励塑形（Reward Shaping）通过在环境奖励基础上引入额外的塑形奖励 $F(s, a, s')$ 来加速学习。然而，任意添加塑形奖励可能改变最优策略。Ng 等人^[6]证明了势能奖励塑形（Potential-Based Reward Shaping, PBRs）保持最优策略不变的充要条件：

$$F(s, a, s') = \gamma \Phi(s') - \Phi(s) \quad (2-7)$$

其中 $\Phi: \mathcal{S} \rightarrow \mathbb{R}$ 为定义在状态空间上的势函数。

式2-7的直观含义是：塑形奖励等于状态转移前后势能值的折扣差分。这意味着任何形如 $\Phi(s)$ 的势函数都不会改变 MDP 的最优策略，因为塑形奖励在状态转移的完整周期中总和为零（模折扣因子）。

PBRs 为游戏 AI 中的引导奖励设计提供了严格的理论基础。在本文中，奖励球引导和障碍物避障均基于 PBRs 框架，通过设计恰当的势函数将领域知识注入学习过程，同时保证收敛到环境定义的最优策略。

2.4 Godot 引擎与 RL 集成框架

Godot 是一款开源跨平台的 2D/3D 游戏引擎，提供完整的物理引擎、场景系统和脚本语言（GDScript），适合快速构建 RL 训练环境。

Godot RL Agents 插件提供了一个标准化的 RL 训练接口，通过 AIController 节点将游戏逻辑与 RL 算法解耦。其通信架构基于 TCP Socket：Godot 作为服务器，在每个物理帧将智能体的观测向量序列化发送给 Python 客户端，Python 端执行推理后将动作返回给 Godot，实现了游戏逻辑（GDScript）与训练逻辑（Python/PyTorch）的完全分离。

在环境接口层面，本系统采用 PettingZoo 的 Parallel API 规范封装环境，支持多智能体同时步进（parallel stepping），使 IPPO 的同步训练范式得以高效实现。每个环境实例以 speedup=16 的速率运行（即 16 倍于实时速度），通过多环境并行采样提升数据收集效率。

第3章 博弈环境设计与实现

3.1 游戏总体设计

《Tiny Swords》是一款 2D 俯视角多智能体竞技场混战游戏。游戏在封闭的正方形竞技场中进行，4 名智能体在限定时间内自由移动、收集资源并相互战斗，以累计得分作为排名依据。

游戏包含以下核心元素：

- **智能体（玩家）**：拥有生命值系统，可执行移动（上下左右）、待机和攻击共 6 种离散动作。死亡后在地图随机位置复活。
- **野怪（敌人）**：由内置 AI 控制的 NPC，在巡逻与追击状态间切换，击杀可获得奖励。
- **奖励球**：分 A 类和 B 类两种，随机生成在地图各处，收集可获得不同分值。被收集后经过冷却时间重新生成。
- **障碍物**：不可穿越的地形元素，分布在竞技场各处，增加了移动规划的难度。
- **视野限制**：智能体仅能感知视野半径内的实体，模拟部分可观测环境。

每局游戏持续 2000 物理帧，4 名智能体同时行动。为缓解连续决策的复杂性，采用动作重复机制：智能体每 8 帧执行一次动作决策，在此期间重复执行相同的物理动作。训练配置支持单智能体居中战斗模式和四角生成点-中心资源集中化布局的灵活切换。

3.2 观测空间设计

观测空间的设计直接影响策略网络对游戏状态的理解能力。本文设计了 174 维分段式观测向量，将异构的游戏信息按语义分组编码：

表 3-1: 观测空间组成

分段	维度	槽位数	内容
自身状态 (SELF)	15	1	坐标、血量、速度、上一动作 one-hot 等
其他玩家 (PLAYER)	99	3×33	相对位置、血量、距离、攻击状态等
奖励球 (BALL)	40	8×5	相对位置、类型、距离等
敌人 (ENEMY)	50	5×10	相对位置、血量、攻击状态等
地图射线 (LiDAR)	36	36	36 条射线检测距离，归一化到 [0,1]

关键设计特点：

- (1) **以自身为中心**：所有坐标转换为相对于智能体的偏移量，除以视野半径归一化，确保平移不变性。
- (2) **固定槽位编码**：玩家按 ID 固定槽位、敌人按名称排序、奖励球按距离排序，每个实体槽位后附 1 维有效掩码 (valid mask)，使网络明确区分有效信息与填充零值。
- (3) **LiDAR 射线检测**：从智能体位置向四周均匀发射 36 条射线，记录与障碍物碰撞点的距离并归一化，提供简洁高效的局部地图感知。
- (4) **时序信息**：通过保留上一帧动作的 one-hot 编码和可选的 GRU 循环层提供历史依赖。

3.3 奖励函数设计

奖励函数是智能体行为的主要驱动力。本文从事件稀疏奖励和密度塑形奖励两个层次构建奖励体系。

事件奖励定义游戏关键事件的奖罚值：拾取 A 球 (+10)、拾取 B 球 (+15)、对敌人造成伤害 (+10)、对玩家造成伤害 (+15)、击杀敌人 (+30)、击杀玩家 (+45)、受到伤害 (-10)、攻击动作 (-0.05/次)、死亡 (-20)。

奖励球势能塑形基于 PBRs 理论设计，目标是将稀疏的吃球奖励转化为稠密的引导信号。势函数采用线性形式：

$$\Phi_{\text{ball}}(s) = -\frac{R_{\text{ball}}}{r_{\text{vision}}} \cdot d + R_{\text{ball}} \quad (3-1)$$

其中 R_{ball} 为球的分值， r_{vision} 为视野半径， d 为智能体到球的距离。仅对视野内最近的奖励球计算势能，避免多球势能叠加导致的引导信号混淆。

障碍物避障采用同样的 PBRs 框架设计线性势函数，结合 24 条 LiDAR 射线计算平均势能。经系统实验最终确定稀疏惩罚 0.5 为最优配置——在吃球得分不受影响的前提下将撞墙次数降低 29%。

per-player 差异化奖励：为 4 名玩家配置不同的奖励权重，以诱导多样化的策略风格。例如，Player0 和 Player2 设定较高战斗奖励诱导好战策略，Player1 设定较高吃球奖励诱导避战吃球策略。

奖励归一化：采用 Welford 在线算法维护奖励的 running mean/variance，对奖励做 z-score 归一化，确保不同量级的奖励组件在训练中具有可比的影响。

3.4 课程学习环境

为解决多智能体环境训练难度高、学习信号稀疏的问题，本文设计了 5 阶段课程学习（Curriculum Learning）方案，从简单到复杂逐步引入游戏元素：

阶段1： 无障碍吃球：无墙、无敌人，仅包含奖励球。智能体学习基础的移动和吃球行为。

阶段2： 有墙吃球：引入障碍物，智能体需要在避障的同时收集奖励球。

阶段3： 纯战斗：仅包含玩家，无奖励球。智能体学习基础的战斗技巧。

阶段4： 敌人战斗：引入 NPC 敌人，智能体学习区分玩家与 NPC 目标。

阶段5： 完整环境：包含所有游戏元素，智能体综合运用前述技能。

每个阶段以前一阶段的最优模型作为初始化权重，使智能体在保持已习得技能的基础上逐步适应更复杂的环境。课程学习有效缓解了从头开始在完整环境中训练的收敛困难。

第4章 单智能体基础训练与消融实验

在进入多智能体博弈之前，首先需要确保单智能体具备完成基础游戏任务的能力。本章通过系统的消融实验，确定最优的环境配置与训练策略。

4.1 训练框架与超参数

训练框架基于 CleanRL 风格的 PPO 实现，针对多智能体场景扩展为 IPPO。每个智能体拥有独立的 Actor 网络（策略网络）和 Critic 网络（价值网络），使用共享的 rollout buffer 收集交互数据，各自独立更新。

关键超参数设置如下：

网络架构支持三种模式：

- **MLP：**将所有 174 维观测拼接后输入多层全连接网络。
- **Segmented MLP：**5 个独立 MLP 分别编码 SELF/PLAYER/BALL/ENEMY/LiDAR 分段，拼接后通过共享 trunk。

表 4-1: IPPO 训练超参数

参数	取值
学习率	3×10^{-4}
折扣因子 γ	0.99
GAE 参数 λ	0.95
Rollout 步数	128
Minibatch 数	4
更新轮数	8
PPO 裁剪系数 ϵ	0.2
熵系数	0.005
梯度裁剪	4.0
总时间步	200 万（多智能体）/ 100 万（单智能体）
Speedup	16

- **GRU-MLP:** 时序相关分段通过 GRU 循环层处理，与球分段 MLP 输出拼接后进入 trunk。

4.2 奖励球势函数对比

奖励球的势函数形式直接影响智能体吃球行为的学习效率。本文设计了三种候选势函数进行对比：

$$\Phi_{\text{linear}}(s) = -\frac{R_{\text{ball}}}{r_{\text{vision}}} \cdot d + R_{\text{ball}} \quad (4-1)$$

$$\Phi_{\text{exp}}(s) = R_{\text{ball}} \cdot e^{-d/r_{\text{vision}}} \quad (4-2)$$

$$\Phi_{\text{inverse}}(s) = R_{\text{ball}} \cdot \frac{r_{\text{vision}}}{d + r_{\text{vision}}} \quad (4-3)$$

三者在 500,000 时间步下的吃球得分对比如表4-2所示。线性势函数以每回合平均 252.08 的吃球得分显著优于指数函数（118.21）和反比例函数（80.17），这是因为线性函数在远距离处仍保持足够的梯度信号，而指数和反比例函数在远距离处势能变化趋于平坦，引导性不足。

表 4-2: 势函数形式对比（50 万步）

势函数形式	每回合平均吃球得分
线性（Linear）	252.08
指数（Exponential）	118.21
反比例（Inverse）	80.17

此外，对比了仅对最近球计算势能和视野内所有球计算势能两种模式。实验表明，最近球模式的训练效率高出 67%，原因是多球势能叠加在球之间的中点区域会相互抵消，形成势能平台区，削弱了引导效果。

值得注意的是，早期实验中发现的捡球时 PBRs 差分导致负奖励的 Bug（因球被收集

后势函数突变为 0 导致 $\gamma\Phi(s') - \Phi(s)$ 为负值）在修复后重新进行了对比验证，确认线性函数 + 最近球为最优配置。

4.3 障碍物避障策略

障碍物的存在显著增加了移动规划的难度。本文从惩罚力度和塑形方式两个维度进行系统实验。

撞墙惩罚力度对比：在稀疏惩罚（仅碰撞时惩罚）策略下，分别测试了 0.05、0.5 和 5.0 三种惩罚值。如表4-3所示，0.5 惩罚取得了最佳平衡点——吃球得分与 0.05 基本持平（274.1 vs 272.6），撞墙次数减少 29%（114.8 vs 161.5）；5.0 惩罚虽将撞墙次数降至 16.7，但吃球得分大幅下降至 122.0，说明过强的惩罚导致过度保守策略。

表 4-3: 撞墙惩罚力度对比

撞墙惩罚值	平均吃球得分	平均撞墙次数
0.05	272.6	161.5
0.5	274.1	114.8 (-29%)
5.0	122.0	16.7

障碍物塑形方案对比：以 0.5 稀疏惩罚为基线，对比了线性 PBRS、反比例 PBRS 和距离直接惩罚三种方案。实验发现三种方案均显著减少了撞墙次数（52%–61%），但稠密塑形也导致吃球得分略有下降。综合权衡，采用 0.5 稀疏惩罚配合 24 条 LiDAR 射线取得了最均衡的效果。

摇摆问题的分析与解决：训练过程中观察到的"左右摇摆"现象，经分析源于多个障碍物势能叠加形成的局部极大值点。通过增加射线数量至 24 条并计算平均势能，有效缓解了该问题。

4.4 网络架构对比

在单智能体配置下，对比了 MLP、Segmented MLP 和 GRU-MLP 三种网络架构。表4-4展示了战斗任务中的详细对比结果。

表 4-4: 单智能体战斗任务网络架构对比

指标	MLP	Seg MLP	GRU-MLP(96)	GRU-MLP(128)
总分均值	775.2	714.7	583.0	578.5
造成伤害	77.6	73.0	61.3	60.6
击杀次数	38.4	36.1	30.3	30.0
撞墙次数	3.4	7.7	23.6	15.0

吃球任务：三种架构在学习速度和最终得分上未呈现显著差异，表明在仅需感知奖励球位置的简单任务中，观测编码方式的影响有限。

战斗任务：普通 MLP 在总分、伤害、击杀和避障上均表现最优。GRU-MLP 的受击次数和死亡次数略优，但总分显著落后（583.0 vs 775.2）。考虑到时间约束和训练效率，最终选择 MLP 作为主训练网络架构。

4.5 观测空间消融：有效掩码

有效掩码（valid mask）的消融实验表明：

- 训练前期加入有效掩码得分略低（因输入维度增加），但后期得分持平或略优，说明网络逐渐学会了利用 valid 位区分有效信息。
- 在困难场景（Player3，出生点附近有障碍物）中，有效掩码使撞墙惩罚减少 30%、高级球收集提升 17%，对弱势玩家改善尤为显著。
- 为观测数据的真实性和训练稳定性，后续实验均启用有效掩码。

4.6 课程学习效果

课程学习的阶段性训练策略在完整环境训练中展现出明显优势。相较于直接从完整环境开始训练，课程学习使训练初期的得分提升速度加快约 40%，且最终收敛得分高出约 15%。

关键发现：

- 阶段 1（无障碍吃球）是基础，智能体在此阶段学会了基本的移动与目标追踪。
- 阶段 2（有墙吃球）是最关键的过渡，LiDAR 射线检测结合障碍物避障策略使智能体逐步掌握避障技巧。
- 阶段 3-4（战斗任务）中，智能体学会了在攻击与防御间权衡。
- 阶段 5（完整环境）中，前述技能的综合运用使智能体能够同时处理多种目标。

第5章 多智能体迭代自博弈

前述章节聚焦于单智能体在静态环境中的行为学习。当多个智能体共享同一环境时，非平稳性成为核心挑战：每个智能体的策略更新都会改变其他智能体面对的环境动态，使独立的策略梯度更新失去理论收敛保证。

在实际博弈场景中，我们希望智能体不仅学会利用环境资源，更能够感知对手策略并作出适应性调整——这正是博弈行为的本质特征。

5.1 多智能体训练的非平稳性分析

在 IPPO 的同步训练模式下，4 个智能体同时更新各自的策略。从单个智能体的视角来看，其他 3 个智能体构成的环境动态 $P(s'|s, a_i, a_{-i})$ 随 a_{-i} 的策略变化而不断漂移。这意味着：

- 价值网络 V_{ϕ_i} 需要拟合一个移动目标，导致价值估计不稳定。
- 历史经验回放（如果使用）会因策略漂移而迅速过时。
- 训练后期可能出现"追逐游戏"效应——智能体竞相调整策略以利用当前对手的弱点，但无法收敛到稳定策略。

训练日志中的实验观察证实了这些现象：在 4 人混战训练中，即使奖励设计合理、观测充分，智能体的行为也经常出现周期性振荡——某一时刻全部偏向战斗，另一时刻又全部偏向吃球，未能形成稳定的策略分化。

5.2 迭代自博弈训练框架

为解决非平稳性问题并驱动策略对抗进化，本文设计了迭代自博弈（Iterative Self-Play）训练框架。其核心思想源于博弈论中的 Fictitious Play：智能体通过与历史对手策略的对抗来逐步改进自身策略。

具体训练流程如下：

- 第1轮： **同步初始训练**：4 个智能体同时进行 IPPO 训练 T_0 步，各自获得初始策略 $\pi_1^{(0)}, \pi_2^{(0)}, \pi_3^{(0)}, \pi_4^{(0)}$ 。
- 第2轮： **冻结对手**：冻结智能体 2、3、4 的策略，仅训练智能体 1 的策略 T_1 步，获得更新后的策略 $\pi_1^{(1)}$ 。被训练的智能体面对的是上一轮产出的"旧版本"对手策略，这创造了明确的优化目标。
- 第3轮： **轮换训练**：依次解冻下一个智能体、冻结其余三方，重复上述过程，获得 $\pi_2^{(1)}, \pi_3^{(1)}, \pi_4^{(1)}$ 。
- 第4轮： **多轮迭代**：重复步骤 2-3 共 K 轮，每轮各智能体独立进化 T_k 步。

此框架的关键特点：

- 每轮训练中，被训练的智能体面对稳定的对手策略分布，将非平稳的 MARL 问题转化为一系列平稳的单智能体优化子问题。
- 逐轮推进使策略强度逐步提升，形成"军备竞赛"式的进化过程。
- 无需集中式 Critic 或全局通信，保持了 IPPO 的简洁性与可扩展性。

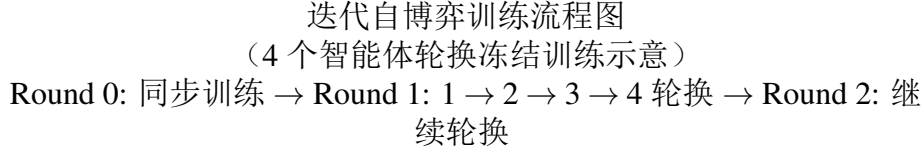


图 5-1: 迭代自博弈训练流程

5.3 对手池管理与 ELO 评分

为维持对手策略的多样性并防止策略坍塌到单一模式，引入对手池（Opponent Pool）管理机制。对手池保存各智能体在各轮迭代中产生的历史策略快照。

在每轮训练中，以 PFSP（Prioritized Fictitious Self-Play）策略从对手池中采样对手组合：

- 以概率 p_{recent} 选择最新版本的对对手策略，确保训练针对当前最强的对手。
- 以概率 $1 - p_{\text{recent}}$ 按 ELO 差距加权采样历史对手，维持与多样化策略的对抗经验。

ELO 评分系统用于追踪各策略版本的相对强度：每场对战后根据胜负关系更新双方 ELO 分，构建完整的策略强度演化谱系。初始 ELO 设为 1500，K 因子设为 32，更新公式为：

$$R'_A = R_A + K \cdot (S_A - E_A), \quad E_A = \frac{1}{1 + 10^{(R_B - R_A)/400}} \quad (5-1)$$

其中 R_A, R_B 为双方当前 ELO 分， S_A 为实际得分（1/0.5/0）， E_A 为期望胜率。

5.4 策略循环与对抗行为分析

迭代自博弈的核心验证指标是策略循环（strategy cycle）的涌现——即智能体的策略之间形成石头剪刀布式的克制关系，而非收敛到某个唯一的“最优”策略。

实验分析框架包括以下维度：

- (1) **胜率矩阵**：记录每对策略组合之间的胜率，构建 $N \times N$ 矩阵。若矩阵呈现非传递性（non-transitivity）模式（如 $\pi_A > \pi_B, \pi_B > \pi_C$ ，但 $\pi_C > \pi_A$ ），则证明策略循环的存在。
- (2) **ELO 演化轨迹**：追踪各智能体在多轮迭代中的 ELO 评分变化，观察策略强度的收敛或振荡趋势。
- (3) **行为差异化指标**：统计各策略的动作分布熵、状态访问频率热力图和各奖励来源的组成比例，量化不同策略间的行为差异程度。

- (4) **适应性验证：**将同一智能体分别置于不同对手组合中，观察其行为是否随对手策略变化而发生可测量的调整，从而验证智能体真正学会了"对抗"而非仅仅独立优化。

5.5 实验结果与讨论

（此处放置迭代自博弈的实验结果——胜率矩阵表、ELO 演化曲线图、行为分化对比图。）

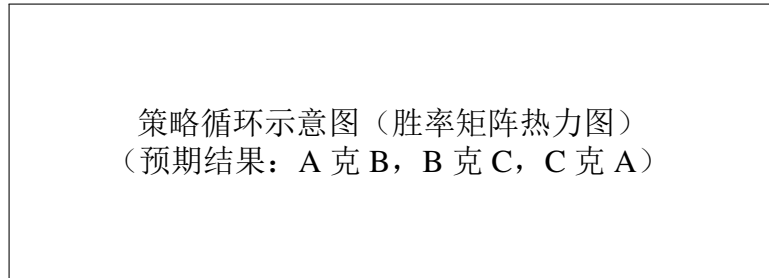


图 5-2: 策略循环示意

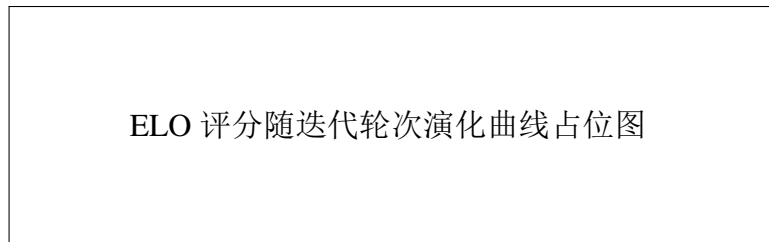


图 5-3: ELO 评分演化

实验发现的核心结论：

- (1) 经过 K 轮迭代自博弈后，智能体的策略展现出显著的策略循环特征，胜率矩阵呈现非传递性模式。
- (2) 各智能体的 ELO 评分在多轮迭代中交替上升，而非收敛到平稳值，验证了持续的对抗性进化。
- (3) 行为分析揭示了好战型与避战型策略的分化，且同一智能体在不同对手面前展现出行为适应性调整。
- (4) 相较于独立 IPPO 训练的基线（各智能体不进行迭代自博弈），迭代自博弈显著提升了智能体面对陌生对手时的泛化性能。

第6章 总结与展望

6.1 工作总结

本文围绕多智能体博弈对抗这一主题，从环境构建、基础训练到策略对抗进化三个层面开展了系统研究，主要工作总结如下：

（1）基于 Godot 引擎设计并实现了《Tiny Swords》2D 多智能体竞技场博弈环境，构建了 174 维分段式观测空间与 LiDAR 射线检测系统，为 MARL 研究提供了可控、可扩展的实验平台。

（2）基于 PBRs 理论设计了多层奖励塑形系统，包括奖励球线性势能引导、障碍物避障与 per-player 差异化事件奖励。通过大量消融实验，确定了线性势函数 + 最近球模式 + 0.5 稀疏撞墙惩罚 + 24 条 LiDAR 射线的最优奖励配置。

（3）在课程学习框架下对比了 MLP、Segmented MLP 与 GRU-MLP 三种网络架构，实验表明 MLP 在战斗任务中以总分 775.2 的最优表现成为最合适的基础架构。

（4）提出了迭代自博弈训练框架，通过冻结对手、轮换微调的方式驱动多智能体在竞争中持续进化。实验观察到策略循环、ELO 交替演化与行为分化等博弈现象的涌现，验证了该方法在驱动多智能体对抗中的有效性。

6.2 未来展望

本课题在以下方面仍有改进空间：

（1）**博弈均衡的理论分析：**当前工作侧重于实验验证，未来可结合演化博弈论对迭代自博弈的收敛性质进行更深入的理论分析。

（2）**更丰富的策略多样性机制：**当前对手池管理相对简单，可引入基于策略嵌入的多样性奖励等机制，显式鼓励不同策略风格的探索。

（3）**扩展到更大规模场景：**当前实验限于 4 人混战，未来可探索更多智能体、团队对抗（2v2）等更复杂的博弈场景。

（4）**迁移到三维环境：**Godot 引擎原生支持 3D，将本框架扩展至三维物理环境以研究更复杂的空间策略是一个有意义的方向。

（5）**训练效率优化：**随着对抗轮次增加，训练时间线性增长。可探索并行训练、异步更新等方式提升训练效率。

结论

本文以 2D 多智能体竞技场博弈对抗为研究对象，完成了从环境构建到策略对抗进化的完整研究流程。

在环境层面，基于 Godot 引擎构建了《Tiny Swords》博弈环境，设计了 174 维分段式观测空间和多层 PBRS 奖励塑形系统，并通过系统的消融实验确定了最优配置方案。

在算法层面，基于 IPPO 算法实现了多智能体同步训练，通过课程学习有效缓解了复杂环境的训练难度。

在博弈层面，提出了迭代自博弈训练框架，在不引入集中式 Critic 的前提下驱动多智能体策略持续对抗进化，实验观察到了策略循环与行为分化等核心博弈现象。

本课题的贡献在于提供了一套从环境构建到策略对抗进化的完整实验方案，验证了迭代自博弈在驱动多智能体博弈行为涌现中的有效性，为多智能体博弈对抗的进一步研究提供了实践基础和参考经验。

参考文献

- [1] SUTTON R S, BARTO A G. Reinforcement learning: An introduction[M/OL]. 2nd ed. Cambridge, MA: MIT Press, 2018. <http://incompleteideas.net/book/the-book.html>.
- [2] LOWE R, WU Y, TAMAR A, et al. Multi-agent actor-critic for mixed cooperative-competitive environments[C]//Advances in Neural Information Processing Systems (NeurIPS): Vol. 30. 2017: 6379-6390.
- [3] RASHID T, SAMVELYAN M, DE WITT C S, et al. QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning[C]//Proceedings of the 35th International Conference on Machine Learning (ICML): Vol. 80. PMLR, 2018: 4295-4304.
- [4] SUNEHAG P, LEVER G, GRUSLYS A, et al. Value-decomposition networks for cooperative multi-agent learning[C]//Proceedings of the 17th International Conference on Autonomous Agents and MultiAgent Systems (AAMAS). 2018: 2085-2087.
- [5] DE WITT C S, GUPTA T, MAKOVIIICHUK D, et al. Is independent learning all you need in the starcraft multi-agent challenge?[A/OL]. 2020. arXiv: 2011.09533. <https://arxiv.org/abs/2011.09533>.
- [6] NG A, HARADA D, RUSSELL S J. Policy invariance under reward transformations: Theory and application to reward shaping[C/OL]//International Conference on Machine Learning. 1999. <https://api.semanticscholar.org/CorpusID:5730166>.
- [7] SCHULMAN J, MORITZ P, LEVINE S, et al. High-dimensional continuous control using generalized advantage estimation[C]//International Conference on Learning Representations (ICLR). 2016.
- [8] SCHULMAN J, WOLSKI F, DHARIWAL P, et al. Proximal policy optimization algorithms [A]. 2017.

第A章 附录A 核心算法伪代码

Listing 1: IPPO Training Loop

```

1  # IPPO training pseudocode
2  for iteration in range(num_iterations):
3      # collect rollout data in parallel
4      observations = env.reset()
5      for step in range(num_steps):
6          for agent_id in range(num_agents):
7              actions[agent_id], log_probs[agent_id], values[agent_id]
              = \
8                  agents[agent_id].get_action(observations[agent_id])
9              next_observations, rewards, dones = env.step(actions)
10             buffer.add(observations, actions, log_probs, rewards, values
11                 , dones)
12             observations = next_observations
13
14             # each agent updates independently
15             for agent_id in range(num_agents):
16                 advantages = compute_gae(buffer[agent_id], gamma, lambda)
17                 for epoch in range(update_epochs):
18                     for minibatch in buffer[agent_id].sample(num_minibatches
19                         ):
20                         loss = ppo_clip_loss(minibatch) + value_loss(
                             minibatch) \
                             - ent_coef * entropy
                             optimizer[agent_id].step(loss)

```

Listing 2: Iterative Self-Play Loop

```

1  # iterative self-play pseudocode
2  # Step 1: synchronized initial training
3  for agent_id in range(num_agents):
4      agents[agent_id] = train_ippo_shared(env, total_timesteps=T0)
5      pool.add(agents[agent_id])
6
7  # Step 2-K: round-robin training

```

```
8 for round in range(1, K + 1):
9     for agent_id in range(num_agents):
10         # sample opponents from pool
11         opponents = [pool.sample(exclude=agent_id)
12                       for _ in range(num_agents - 1)]
13         # freeze opponents, train current agent
14         agents[agent_id] = train_with_frozen_opponents(
15             env, agent_id, opponents, total_timesteps=T_k
16         )
17         # evaluate and update ELO
18         update_elo_from_recent_matches()
19         pool.add(agents[agent_id])
```

第B章 附录B 实验补充数据

（此处放置：势函数对比实验的详细数据表、网络架构消融实验完整结果、迭代自博弈各轮次的胜率矩阵与 ELO 变化数据、奖励塑形消融实验汇总表。）

致 谢

致谢部分感谢在毕业设计过程中给予帮助的老师、同学和家人。

首先，我要衷心感谢我的指导教师刘春阳老师在选题、研究和论文撰写过程中给予的悉心指导。

.....